Assignment 5

1. Student Marks Input Validation

Write a Python program that accepts marks of 5 students from the user and stores them in a list.

Requirements:

- If the user enters a non-numeric value, handle the exception.
- If the marks are less than 0 or greater than 100, raise a `ValueError`.
- Display the final valid list of marks.
- Use `try-except-else-finally`.

2. Simple Calculator with Multiple Exceptions

Create a menu-driven calculator program for:

- addition
- subtraction
- multiplication
- division

Requirements:

- Accept two numbers from the user.
- Handle:
  - `ValueError` for invalid input
  - `ZeroDivisionError` for division by zero
  - invalid menu choice using `raise`
- Show proper messages for each case.

3. File Reader Utility

Write a Python program that asks the user for a filename and reads its contents.

Requirements:

- Handle `FileNotFoundError` if file does not exist.
- Handle `PermissionError` if access is denied.
- Use `finally` block to print: `"File operation completed."`
- Count and display:
  - number of lines
  - number of words
  - number of characters

4. Bank Withdrawal System using User-Defined Exception

Create a bank account program.

Requirements:

- Accept account balance and withdrawal amount.
- If withdrawal amount is greater than balance, raise a custom exception named `InsufficientBalanceError`.
- Display proper message.
- If withdrawal is valid, show remaining balance.

5. Employee Age Verification

Write a program to accept employee name and age.

Requirements:

- If age is below 18, raise an exception with a suitable message.
- If input age is non-numeric, handle it.
- If age is valid, print that the employee is eligible.

6. Online Shopping Cart – Exception Chaining

Write a program for an online shopping cart where quantity should be entered as an integer.

Requirements:

- Accept quantity from user.
- If conversion fails, catch the original exception and raise a new exception using exception chaining.
- Example: convert `ValueError` into `TypeError` or custom exception using `raise ... from ...`

Expected idea:

- Input: `"abc"`
- Original exception: `ValueError`
- New exception: `"Invalid quantity entered"`

7. Password Validation System

Write a program that accepts a password from the user.

Requirements:

Password must satisfy:

- at least 8 characters
- at least 1 digit
- at least 1 uppercase letter

If any rule fails:

- raise a suitable exception
- show meaningful message

Additional requirement:

- create a custom exception class named `InvalidPasswordError`

8. List Index Access Program

Write a Python program that creates a list of 5 elements and asks the user for an index to access an element.

Requirements:

- Handle `IndexError` if index is out of range.
- Handle `ValueError` if the index entered is not an integer.
- Always print `"Program ended"` using `finally`.

9. Data Conversion Report Generator

Write a program that reads a list of strings:

```
data = ["10", "20", "abc", "40", "xyz", "60"]
```

Requirements:

- Convert valid values into integers and store them in a new list.
- Handle invalid values using exception handling.
- Maintain a separate error log list for invalid entries.
- At the end, print:

    - valid integer list
    - invalid values list

10. Mini Admission System (Comprehensive Question)

Create a program for student admission.

Requirements:

- Accept:
    - student name
    - age
    - percentage
    - course name
- Handle invalid numeric input.
- Raise exception if:
    - age is less than 16
    - percentage is not between 0 and 100
    - course name is blank
- Use at least one custom exception.
- Use at least one example of exception chaining.
- Use `finally` block.
- If all data is valid, print admission confirmation.

Special Programming Tasks on Important Concepts

A. Separate Practice on Raised Exception

Write a program that accepts a number from the user and raises a `ValueError` if the number is negative.

Example:

- Input: `-5`
- Output: `"Negative numbers are not allowed"`

B. Separate Practice on User-Defined Exception

Create a custom exception called `UnderAgeError` and use it in a voting eligibility program.

Condition:

- If age is less than 18, raise `UnderAgeError`

C. Separate Practice on Exception Chaining

Write a program that tries to convert a string into an integer.
If conversion fails:

- catch the exception
- raise a new exception using:

```
raise TypeError("Custom conversion failed") from e
```